

Gold Monitor

Deployment & Development Guide

Project	goldmonitor
Repository	github.com/pyaephyoaung-pa/goldmonitor
Hosting	Vercel (webhook) + GitHub Actions (polling)
Domain	goldmonitor-xi.vercel.app
Last Updated	May 2026

Table of Contents

1. Project Architecture
2. File Structure
3. Adding New Features (Step-by-Step)
4. Git Push & Deployment Flow
5. Vercel Webhook Deployment
6. GitHub Actions Workflows
7. Environment Variables
8. Testing Before Push
9. Rollback Procedure
10. Troubleshooting

1. Project Architecture

Gold Monitor has two parallel systems that respond to Telegram bot commands:

Component	How It Works	Response Time
Vercel Webhook	Telegram sends POST to goldmonitor-xi.vercel.app/api/webhook	Instant (~1s)
GitHub Actions Polling	bot_commands.yml runs every 5 min, checks for new messages	Up to 5 min delay
Gold Monitor	gold_monitor.yml runs every 8 min, checks price & sends alerts	Every 8 min

Note: Both webhook and polling handle the same commands. The webhook provides instant replies; polling acts as a fallback if the webhook is down.

2. File Structure

File	Purpose
api/webhook.py	Vercel serverless function - instant Telegram responses
bot_commands.py	GitHub Actions polling - checks Telegram every 5 min
gold_monitor.py	Price checker - runs every 8 min, sends alerts
storage.py	GitHub Gist-based storage (price history, portfolio, state)
predictor.py	Technical analysis & price prediction
vercel.json	Vercel deployment config (routes, includeFiles)
.github/workflows/gold_monitor.yml	Cron: every 8 min - price monitoring
.github/workflows/bot_commands.yml	Cron: every 5 min - command polling
requirements.txt	Python dependencies
setup_webhook.py	One-time: register Telegram webhook URL
setup_gist.py	One-time: initialize GitHub Gist storage

3. Adding New Features (Step-by-Step)

When adding a new bot command (e.g. `/balance`), you need to update multiple files to keep webhook and polling in sync.

Step 1: Add storage functions (if needed)

If your feature needs persistent data, add functions to `storage.py` first.

```
def get_balance():
    data = _read_gist_file("balance.json")
    return data or {"balance": 0}
```

Step 2: Add command handler

Write your command handler function. This same logic goes into BOTH files:

a) `bot_commands.py` — add the handler function + register in `COMMANDS` dict

b) `api/webhook.py` — add the same handler function + register in `COMMANDS` dict

```
def cmd_balance(chat_id: str, args: str):
    # Your command logic here
    send_message("Your balance: ...", chat_id)

COMMANDS = {
    ...
    "/balance": cmd_balance, # Add this line
}
```

Important: Always update BOTH `bot_commands.py` and `api/webhook.py` with the same command. Otherwise, commands will only work via one channel.

Step 3: Update `/help` command

Add your new command to the help text in `cmd_help()` in both files.

4. Git Push & Deployment Flow

After making changes, push to GitHub. Deployment happens automatically:

Step	Action	What Happens
1	<code>git add .</code>	Stage all changed files
2	<code>git commit -m "Add /balance command"</code>	Create commit locally
3	<code>git push origin main</code>	Push to GitHub
4	(automatic)	Vercel detects push, rebuilds <code>webhook.py</code> (takes ~20-30 seconds)

5	(automatic)	GitHub Actions picks up new workflow files on next scheduled run
---	-------------	--

Complete git commands:

```
cd ~/Projects/goldmonitor
git add .
git commit -m "feat: add /balance command"
git push origin main
```

Tip: Use conventional commit messages: `feat:` for new features, `fix:` for bug fixes, `chore:` for config changes.

5. Vercel Webhook Deployment

How it works:

1. Vercel watches the `main` branch on GitHub.
2. On every push, Vercel builds `api/webhook.py` as a serverless function.
3. The function is deployed to `goldmonitor-xi.vercel.app`.
4. Telegram sends webhook POST requests to `/api/webhook`.

Key configuration (vercel.json):

```
{
  "builds": [{
    "src": "api/webhook.py",
    "use": "@vercel/python",
    "config": {
      "includeFiles": ["storage.py", "predictor.py"]
    }
  }]
}
```

Important: `webhook.py` uses `sys.path.insert()` to import `storage.py` and `predictor.py` from the parent directory. If you add a new Python module that `webhook.py` needs, you must also add it to the `includeFiles` array in `vercel.json`.

Verify deployment:

After pushing, visit `https://goldmonitor-xi.vercel.app/api/webhook` in your browser. A GET request should return:

```
{"status": "ok", "service": "Gold Monitor Telegram Webhook", ...}
```

6. GitHub Actions Workflows

Workflow	Schedule	File	Purpose
Gold Price Monitor	Every 8 min Mon-Fri 8AM-12AM Sat 10AM-5PM (BKK)	gold_monitor.yml	Fetch price, store history, send alerts
Bot Commands Polling	Every 5 min 24/7	bot_commands.yml	Poll Telegram for commands, process & reply

To change schedule intervals, edit the `cron:` expressions in the YAML files. Cron times are in UTC (Bangkok = UTC+7).

7. Environment Variables

Secrets are stored in two places. Both must have the same values:

Variable	GitHub Secrets	Vercel Env Vars	Description
TELEGRAM_BOT_TOKEN	Yes	Yes	Bot API token from @BotFather
TELEGRAM_CHAT_ID	Yes	Yes	Your Telegram chat ID
GIST_GITHUB_TOKEN	Yes	Yes	GitHub token for Gist storage
GIST_ID	Yes	Yes	GitHub Gist ID for data
TWELVE_DATA_API_KEY	Yes	Yes	Gold price API key
WEBHOOK_SECRET	No	Optional	Telegram webhook verification

8. Testing Before Push

Local syntax check:

```
python -c "import py_compile; py_compile.compile('bot_commands.py', doraize=True)"
python -c "import py_compile; py_compile.compile('api/webhook.py', doraize=True)"
python -c "import py_compile; py_compile.compile('storage.py', doraize=True)"
```

Test webhook locally:

```
# Test the handler imports work
cd api && python -c "import webhook; print('OK')" && cd ..
```

Test after deployment:

```
# Health check
curl https://goldmonitor-xi.vercel.app/api/webhook

# Send a test command in Telegram
/help
```

9. Rollback Procedure

If a deployment breaks the bot, rollback quickly:

Option A: Git revert

```
git log --oneline -5      # Find the bad commit
git revert HEAD           # Revert last commit
git push origin main      # Push revert (triggers redeploy)
```

Option B: Vercel dashboard

Go to vercel.com > [goldmonitor](#) > Deployments. Find the last working deployment and click the three-dot menu to redeploy it.

10. Troubleshooting

Problem	Cause	Fix
Bot not responding to any commands	Webhook import error (500 on Vercel)	Check Vercel logs at vercel.com > Logs. Usually a missing import or new file not in includeFiles.

Commands work slow (5 min delay)	Webhook is down, only polling works	Check webhook health: GET /api/webhook. Fix and redeploy.
ModuleNotFoundError in Vercel logs	New .py file not in vercel.json includeFiles	Add file to includeFiles array in vercel.json.
/price shows API error	Twelve Data API quota exceeded	Check daily quota (800/day). Reduce monitor frequency or wait until reset.
GitHub Actions not running	Workflow disabled or cron syntax error	Go to repo > Actions tab. Re-enable workflow if disabled.